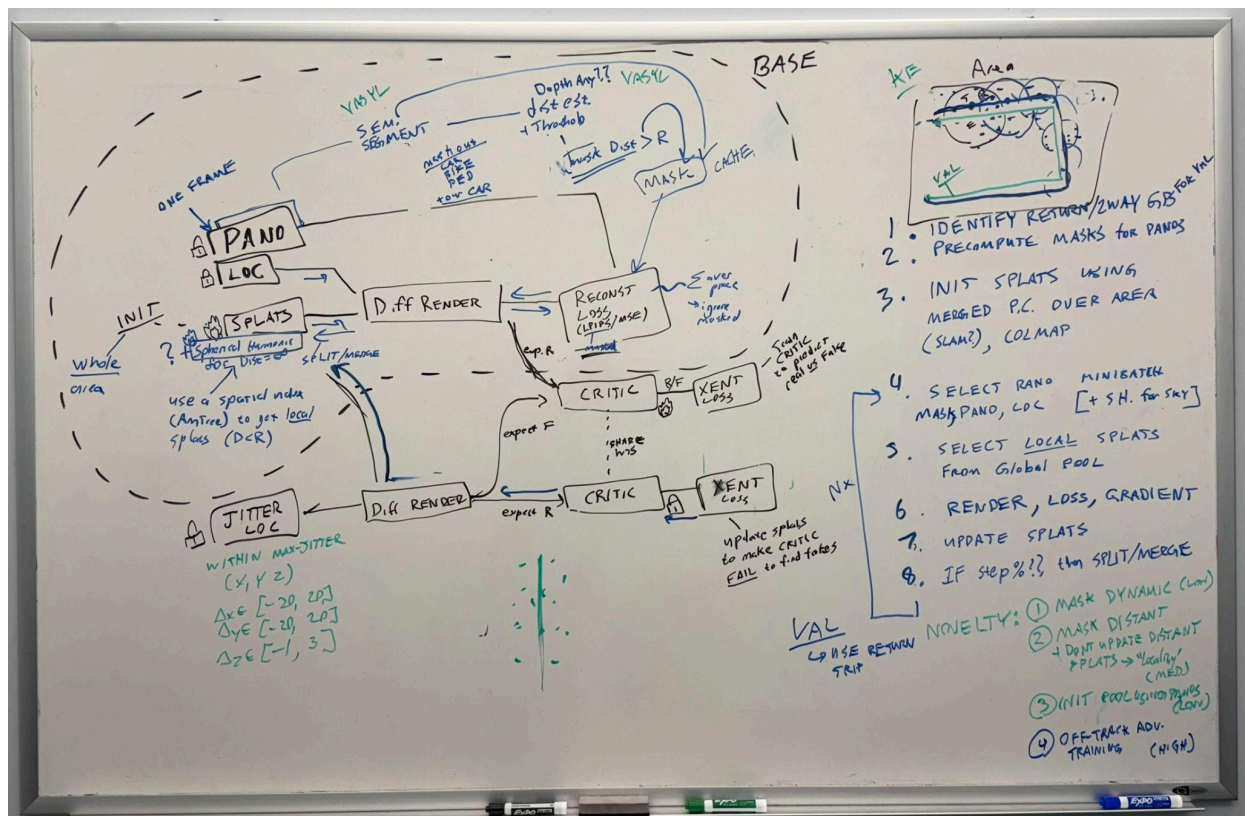




Can you hear me? Okay, so I'm gonna paste in an image that we drew on the whiteboard. The idea is that we're trying to do Gaussian splatting, but make it robust to having points that aren't on the original camera path. So the issue with regular Gaussian splatting is traditionally you would take images from a bunch of point of views, but we can't densely sample the views. Instead, we're just walking and recording with a 360 camera. The problem with that is we can do a good job fitting splats that look correct when we're on that same path, but when we perturb our location so that we're off the path, like two meters to the right, two meters to the left, a meter above, whatever it is, all of a sudden the splats look like garbage when we're off the path. There was nothing during the optimization process for fitting the splats that caused us to look good from those new locations. During the optimization process, we basically used the mean squared error between the splat renderings and the video, right? So the idea here is we wanna make a system so that we can fit the splats and have it be more robust so that it still looks realistic when we're off the path. And there are a couple of ways to do that. The idea that we have in mind is to use a critic, similar to a GAN. So what we'll do is we'll, the traditional thing, it tries to sample images from the original path. We're gonna do that plus 50% of the time or something, we're going to sample a viewing location that was off of the original path for which there is no actual image, right? So we'll jitter the XY location of a real image and then we'll render it using the splats. Now, what we wanna do is we're going to train a critic by showing it both points that were rendered from the original path and points that weren't rendered from the original path and ask it to predict which is which. And we'll use cross-entropy loss on the critic to teach it how to figure out the difference between things rendered from where we expect it to do well and things

rendered from where it does poorly. Then we're going to take that critic, and we're going to update the Gaussian splats. We're going to use one minus the score, or we're going to use the negative of the cross-entropy loss of that critic as a loss to drive the process of fitting the splats. So in essence, we're doing adversarial or training of where the Gaussian splats should be. And we're mixing that in with the traditional training. So this is the idea. We'll paste in the whiteboard figure, and I want you to generate a clear as possible description of the ideas being depicted. Just an FYI, another idea that's being depicted here is that we might want to mask out certain parts of the original image because those things are dynamic. Pedestrians, bicycles, the hood of the car that we're using to track the camera. And the same idea, you shouldn't be able to tell the difference between the masked out parts and the real parts, for example. That's not shown here in the process, but it's there. Another thing that's going to be shown in this whiteboard document is how we validate this. When we capture our data, we walk along one path, then we turn around and walk back, but we're on the other lane of the road or the other side of the street. So really, we're going to validate this by seeing how well we can reconstruct the images from the other side of the street where we actually have data, but we don't use that data to fit the splats. Okay, now I'm going to paste the picture.

Problem Setup

You're doing **Gaussian Splatting from sparse trajectory data** (a single walking path with a 360° camera), and the core issue is:

- Training only enforces correctness **on the observed trajectory**
- As soon as you move **off that path (lateral / vertical perturbations)**:
 - geometry becomes inconsistent
 - splats produce artifacts ("garbage views")

So the goal is:

Learn splats that are **robust to off-trajectory viewpoints**, even without ground-truth images there.

Core Idea: Adversarial View Robustness

You augment standard Gaussian splatting with a **critic (discriminator-like network)** that enforces realism beyond the training path.

Pipeline Overview

1. Standard Inputs

From each frame:

- **PANO** (360 image)
- **Camera pose (LOC)**

You maintain:

- A global set of **Gaussian splats**
 - A **differentiable renderer**
-

2. Two Types of View Sampling

A. On-path views (supervised)

- Real camera poses
 - Render splats → compare to real image
 - Loss:
 - Reconstruction (L1 / MSE / LPIPS)
-

B. Off-path views (unsupervised)

- Sample perturbed camera poses:
 - jitter in (x, y, z)
 - e.g. $\pm 2\text{m}$ lateral, $\pm 1\text{m}$ vertical
- Render splats → **no ground truth exists**

This is where the critic comes in.

3. Critic (Adversarial Signal)

You train a **binary classifier**:

Input:

Rendered images

Labels:

- **Real (on-path renders)** → should look correct
- **Fake (off-path renders)** → currently look bad

Loss:

- Cross-entropy

So the critic learns:

“Does this rendering look like something that could have come from the real trajectory?”

4. Updating the Splats (Adversarial Step)

Now you **flip the objective**:

- Use the critic’s output as a signal to update splats
- Encourage off-path renders to **fool the critic**

Effectively:

- Splats minimize:
 - Reconstruction loss (on-path)
 - adversarial loss (off-path)

So:

The splats are trained to produce images that the critic cannot distinguish from valid (on-path) views — even when off-path.



Training Loop

Each iteration:

1. Sample a frame
 2. With some probability (e.g. 50%):
 - Use original pose
 - OR jitter pose
 3. Render image from splats
 4. Compute:
 - Reconstruction loss (if on-path)
 - Critic classification loss
 5. Update:
 - Critic → classify real vs off-path
 - Splats → minimize:
 - reconstruction loss
 - adversarial loss (via critic)
-



Masking Dynamic Content

You also include **masking of unreliable regions**:

Examples:

- pedestrians
- bicycles
- car hood
- dynamic objects

Purpose:

- Prevent splats from trying to explain **non-static geometry**
- Avoid corrupting the adversarial signal

Additional idea:

You may also:

- Train critic to ignore masked regions
 - Or enforce consistency between masked/unmasked areas
-



Spatial Locality (Important Detail)

From the diagram:

- You **don't render all splats every time**
- Instead:
 - Use a spatial index (e.g., KD-tree)
 - Select **local splats near the camera**

This improves:

- efficiency
 - locality consistency
-



Splat Management

You also include:

- **Split / merge operations**
 - Periodic updates to:
 - refine density
 - avoid degenerate splats
-



Validation Strategy (Very Nice)

This part is particularly strong:

Data collection:

- Walk forward on one side of the street
- Walk back on the other side

Training:

- Use only **forward path**

Validation:

- Evaluate on **return path (unseen viewpoints)**

So you get a **real ground-truth test** of:

off-trajectory generalization



Key Insights

1. You are turning view generalization into a GAN problem

Instead of:

- supervising geometry explicitly

You:

- **learn a realism prior over renderings**
-

2. Off-path views become “hard negatives”

The critic identifies:

- geometric inconsistencies
 - missing structure
 - view-dependent artifacts
-

3. The splats learn implicit 3D consistency

Because:

- the only way to fool the critic is to become **view-consistent**
-



Potential Challenges (worth noting)

- Critic may overfit to trivial cues (blur, aliasing)
- Training instability (GAN-like behavior)
- Need careful balance:

- reconstruction vs adversarial loss
 - Pose jitter distribution is critical:
 - too large → unrealistic supervision
 - too small → weak signal
-



One-Sentence Summary

You augment Gaussian splatting with an adversarial critic trained to distinguish on-path vs off-path renderings, and use that signal to force the splats to produce geometrically consistent and realistic views even from unseen viewpoints.

If you want, I can help you turn this into a paper-style method section or suggest concrete loss formulations / architectures for the critic.

Since we're using Gaussian splatting, is mode collapse really likely to be a problem? You mentioned training instability. I question this. Hit the check.

Good instinct to question that — you're right to be skeptical. Let's be precise.



Short Answer

Classic GAN-style mode collapse is very unlikely here.

But **training instability is still a real concern**, just for *different reasons* than in image-generation GANs.



Why Mode Collapse Is Unlikely

Mode collapse happens when:

The generator maps many inputs → the same output

That doesn't really apply here because:

1. Your “generator” is not a free network

- It's a **structured scene representation (Gaussian splats)**
- It's constrained by:
 - geometry
 - camera poses
 - projection consistency

So it **can't arbitrarily collapse outputs** without breaking reconstruction.

2. You have strong supervised anchors

- On-path views have **direct reconstruction loss**
- That enforces:
 - spatial correctness
 - appearance fidelity

So the system cannot “cheat” globally.

3. The mapping is multi-view consistent

Each splat affects many views → this creates:

- strong coupling across viewpoints
- implicit regularization

This is very different from a GAN generator that outputs independent images.

So What *Could* Go Wrong?

Not mode collapse, but **adversarial imbalance / degenerate equilibria**.

Here are the real risks:

1. Critic learns shortcuts (biggest issue)

The critic might distinguish views based on:

- blur / sharpness differences
- rendering artifacts (aliasing, density)
- FOV distortions
- edge statistics

Instead of:

true 3D inconsistency

Result:

- Splats optimize for **appearance tricks**, not geometry
 - Off-path views may still be wrong geometrically
-

2. Weak gradient signal to splats

Unlike GANs:

- Your generator is **not very expressive locally**
- Splats update through:
 - position
 - covariance
 - color

So:

The critic may detect errors the splats cannot easily fix

Result:

- Training stalls
 - Adversarial loss becomes noisy or uninformative
-

3. Local “cheating” instead of collapse

Instead of collapsing, splats might:

- inflate covariances (blurry blobs)
- smear geometry

- reduce high-frequency detail

Why?

Because “blurry but consistent” can fool the critic

4. Instability = oscillation, not collapse

What you might see:

- critic improves → splats degrade
- splats adapt → critic shifts again

But:

- no collapse to a single mode
 - more like **loss oscillation or slow convergence**
-

5. Distribution mismatch (subtle but important)

You’re training the critic on:

- on-path renders (good)
- off-path renders (bad, early on)

But over time:

- off-path improves → distribution shifts

This creates a **moving target problem**, which can destabilize training.



Bottom Line

You’re correct:

✗ Mode collapse (in the classic GAN sense) is not the main risk.

But:

⚠️ Instability can still arise from **critic misuse, weak gradients, and degenerate geometric solutions**.



What Would I Watch For?

If you run this, monitor:

- Off-path sharpness vs on-path sharpness
 - Covariance growth of splats
 - Critic accuracy (if it goes to ~100%, bad sign)
 - Whether adversarial loss actually improves validation views
-



If You Want to Make It More Stable

Without overhauling your idea:

- Use **feature matching loss** (not just CE)
 - Limit critic capacity
 - Regularize splat size (avoid blur cheating)
 - Keep strong weight on reconstruction loss
-



Final Take

You're not building a GAN in the usual sense — you're adding a **consistency prior via discrimination**.

So the failure mode is not:

“everything looks the same”

It's:

“everything looks plausibly real but geometrically wrong”

If you want, I can sketch a more principled formulation (e.g., energy-based or contrastive instead of GAN-style) that might align even better with your setup.

Explain what you mean by energy-based or contrastive.

Great question — this is exactly the right place to push the idea further. What you're currently doing is *GAN-flavored*, but your problem actually fits **energy-based** or **contrastive formulations** more naturally and often more stably.

I'll explain both in the context of *your exact setup*.

1. Energy-Based View (EBM)

Core Idea

Instead of a classifier that says:

“real vs fake”

you learn a function:

$E(x)$ = “how bad / unrealistic is this image?”

Lower energy = more plausible

In Your Setting

Let:

- x_{on} = render from real trajectory
- x_{off} = render from jittered pose

You train a network $E(x)$:

- Want:
 - $E(x_{\text{on}}) \rightarrow \text{low}$
 - $E(x_{\text{off}}) \rightarrow \text{high}$

Loss (ranking / margin style):

$$\mathcal{L} = \max(0, E(x_{\text{on}}) - E(x_{\text{off}}) + m)$$

No probabilities, no sigmoid, no cross-entropy.



How Splats Use It

Instead of “fooling a discriminator”, splats minimize:

$$\mathcal{L}_{\text{splat}} = E(x_{\text{off}})$$

So:

“Make off-path renders have low energy (i.e., look realistic)”



Why This Is Better for You

- No need to learn a **decision boundary**
- No saturation (a big GAN issue)
- Provides **dense gradients** (not just classification signal)
- Fits your goal:
 - you don’t care about “real vs fake”
 - you care about **plausibility**



Intuition

You're learning:

a **visual consistency prior over renderings**

instead of:

a binary judge



2. Contrastive Learning View



Core Idea

You don't classify images.

You say:

"These two should be similar; these two should be different."



In Your Setup

You can form pairs:

Positive pairs:

- same location, small perturbation
- or nearby viewpoints

Negative pairs:

- off-path vs on-path
 - or inconsistent renders
-



Example

Let:

- $f(x)$ = embedding of image

Then enforce:

$$\text{sim}(f(x_{\text{on}}), f(x'_{\text{on}})) \gg \text{sim}(f(x_{\text{on}}), f(x_{\text{off}}))$$

Using something like InfoNCE:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(x_{\text{on}}, x'_{\text{on}})/\tau)}{\sum \exp(\text{sim}(x_{\text{on}}, x_j)/\tau)}$$

How Splats Use It

Splats are updated so that:

- off-path renders become **closer in feature space** to on-path renders

Why This Fits Your Problem

You don't actually have "fake images" — you have:

"less consistent views"

Contrastive learning captures that better:

- enforces **multi-view consistency**
- doesn't rely on fragile binary labels
- encourages **geometry-aware similarity**

GAN vs Energy vs Contrastive (For You)

Method

What it learns

Risk

Fit for your problem

GAN (yours)	decision boundary	shortcut learning	👍 good, but fragile
Energy-based	plausibility score	easier to stabilize	✅ very good
Contrastive	consistency in feature space	needs good sampling	✅ very good



Key Insight

Your problem is NOT:

“Is this image real?”

It is:

“Is this image **consistent with a valid 3D scene?**”

That’s why:

- GAN framing = slightly misaligned
 - EBM / contrastive = more natural
-



Concrete Recommendation

If I were implementing your system:

Minimal change (best first step):

👍 Replace cross-entropy critic with **energy ranking loss**

- same architecture
 - different objective
 - better gradients
-

Next step (stronger):

👉 Add **feature matching / contrastive loss**

- compare embeddings of:
 - nearby views
 - jittered views
-



One-Line Summary

- **GAN**: “fool the classifier”
 - **EBM**: “lower the realism energy”
 - **Contrastive**: “be consistent with nearby valid views”
-

If you want, I can propose a *very concrete loss function + training loop* tailored to Gaussian splatting (including how to sample poses and weight losses).

I don't understand either one. First of all, I don't really see the difference between what you're calling contrastive and the GAN. It's just, are you trying to train the GAN to predict high versus low instead of yes versus no? I just don't get the difference there. It seems like it's not going to fix any kind of stability issue. The other one about energy-based, Yeah, that was the energy-based one that you suggested. Okay, that didn't make sense. The other one about contrastive, contrastive between what? The nature of the way things change when you sample along the path is going to be fundamentally different than the way that they change when you go perpendicular to the path. So I just don't see how this is going to work for contrastive with the check.

Good — this is exactly the right pushback. Let me reset and explain this in a way that actually maps to your system, not abstract ML buzzwords.



First: You're right about one thing

“Isn't contrastive just high vs low instead of yes vs no?”

Yes — superficially.

But the *difference is not the output*, it's **what signal the model is trained to use**.

That's the key.

Why your current GAN framing is risky (precisely)

Your critic is trained to answer:

“Was this rendered from an on-path pose or an off-path pose?”

That is **not the same as**:

“Is this geometrically correct?”

This creates a loophole:

The critic can win by detecting:

- motion patterns
- parallax differences
- distortion statistics
- camera distribution biases

without ever learning 3D correctness

So the splats learn:

“Make off-path renders *look statistically like on-path renders*”

—not:

“Make them geometrically correct”

Your objection about contrastive is valid

You said:

“Things change differently along-path vs perpendicular — so contrastive won’t work.”

Correct — if you set it up naively.

So the key is:

Contrastive should NOT compare *along-path vs off-path directly*

That would indeed fail.

What contrastive *should* mean here

Not:

“on-path vs off-path”

But:

“locally consistent vs inconsistent views”

The correct contrastive framing for YOUR problem

Think in terms of view consistency

If the scene is correct:

- small viewpoint changes → **predictable image changes**
- geometry induces **smooth, structured variation**

If splats are wrong:

- small perturbations → **weird artifacts**
 - inconsistencies appear
-

 **So define contrastive pairs like this:**

Positive pairs (should match):

- render at pose p
- render at pose $p + \epsilon$ (small jitter)

These should be **consistent**

Negative pairs (should NOT match):

- render at pose $p + \epsilon$
 - render at pose $p + \delta$ where:
 - δ exposes geometric errors (e.g. lateral shift)
-

 Important:

You are NOT saying:

“off-path is wrong”

You are saying:

“if geometry is correct, nearby views should agree in structure”

 This is the key difference vs GAN

GAN critic learns:

- **dataset bias** (on-path vs off-path distribution)

Contrastive learns:

- **geometry-induced consistency**

That's a completely different signal.



Why this matters

Your GAN can succeed even if:

- geometry is wrong
- but textures/statistics match on-path views

Your contrastive setup forces:

local multi-view consistency

Which is exactly what you want.



What about Energy-Based?

Let me clarify that more concretely too.

Your current loss (GAN-style):Critic:

Your current loss (GAN-style):

Critic:

$$\text{CE}(D(x), \text{on/off})$$

Splats:

fool D

Energy version (what I meant, concretely):

Instead of:

classify on vs off

You train:

$$E(x) = \text{"how inconsistent does this look?"}$$

Train it like:

$$E(x_{\text{good}}) < E(x_{\text{bad}})$$

Where:

- good = on-path render
- bad = perturbed render

The key difference

No sigmoid, no probability.

So:

- no saturation
- no "critic gets too confident → gradients die"

Why this matters practically

GAN problem:

- if critic gets too good → splat gradients vanish

Energy model:

- always gives **relative signal**

 **Let me be blunt**

You are right that:

“just renaming things won’t fix instability”

Correct.

The **real issue** is:

 **Your current signal:**

“Does this look like it came from the training trajectory?”

 **What you actually want:**

“Is this consistent with a valid 3D scene?”

The core fix (independent of method)

No matter what (GAN / energy / contrastive), you need:

The critic to detect:

- multi-view inconsistency
- geometry violations
- parallax errors

NOT:

- trajectory membership
-

If you keep your GAN (totally reasonable)

Then I would change ONE thing:

 **Don’t label by on-path vs off-path**

Instead:

- Positive:
 - real images OR high-quality renders
- Negative:
 - deliberately corrupted renders
 - wrong depth ordering
 - shuffled splats
 - exaggerated parallax

Now the critic learns:

“what broken geometry looks like”



Final Take

- Your skepticism is correct
- Contrastive is not magic unless defined properly
- Energy-based is mostly about **better gradients**, not different supervision

The real issue is:

What signal teaches the splats what “wrong geometry” is?

If you want, I can propose a **very concrete minimal change** to your current pipeline that fixes the biggest conceptual issue without redesigning everything.

Okay, your ideas sound great, except there is no training signal to train an energy or contrastive system. There's no signal there. We don't have enough to build those. We only can do on-path versus off-path. There's no additional source of information for this contrastive or energy-based system to be built on.

Yes — that's the right conclusion.

I was being too loose before. **Energy-based and contrastive do not magically create a new training signal.** If the only information you actually have is:

- on-path samples
- off-path samples generated by pose perturbation

then **that is the only supervision source** available to the auxiliary model, regardless of whether you package it as:

- binary classification
- energy ranking
- contrastive embedding

So your objection is correct:

without extra structure, these are mostly reparameterizations of the same weak signal.

That is the key point.

What that means

If you train anything using only:

- “this came from the original path”
- “this came from a perturbed path”

then the model can still learn the shortcut:

“detect trajectory membership”

instead of:

“detect geometric wrongness”

Changing the loss does **not** fix that by itself.

So was the GAN idea wrong?

Not necessarily. It just means the real question is not:

GAN vs energy vs contrastive?

It is:

Is on-path vs off-path a strong enough supervisory distinction to teach the right thing?

That is the real issue.

And honestly, I think the answer is:

- **maybe yes, but only if your perturbation design is very careful**
- otherwise the critic will learn nuisance cues

Where the signal can still come from

Even though there is no extra external supervision, there is still an **implicit assumption** you are trying to use:

a correct static 3D representation should render plausibly not only on the path, but also under small nearby viewpoint changes

That is not a labeled truth source, but it is a structural prior.

So your method is really trying to inject this prior:

- on-path renders are anchored by reconstruction
- off-path renders are regularized to remain plausible nearby

That can work, but it is fundamentally a **prior-driven method**, not a fully supervised one.

The honest limitation

Your auxiliary loss cannot prove an off-path image is “correct,” because there is no ground truth there.

At best, it can encourage:

- smoothness under local viewpoint perturbation
- consistency with the appearance statistics of observed views
- avoidance of obvious off-path artifacts

That is weaker than true supervision.

So I would reframe the method this way:

We are not learning correctness off-path. We are discouraging catastrophic failure off-path.

That is a much more accurate claim.

The strongest version of your current idea

Given your constraints, I think the cleanest statement is:

- Use standard reconstruction on-path
- Add an adversarial regularizer on small off-path perturbations
- Interpret it as a **local robustness prior**
- Keep perturbations small enough that “should still look plausible” is defensible

That makes the method much more coherent.

Because if you perturb too far, the critic is being asked to enforce something you do not know to be true.

So what should you believe now?

I'd summarize it like this:

- You are right that energy/contrastive do not give you new supervision.
- They only change how the same weak supervision is used.
- The central bottleneck is not the loss family.
- The central bottleneck is whether **on-path vs nearby-off-path** is a meaningful enough prior to improve generalization.

I think that is the real check.

My corrected take

Your original GAN-style idea is actually a reasonable first method **given the data you have**.

But its role is modest:

- not to recover true unseen-view correctness
- but to bias the splats away from brittle, path-specific solutions

That is a believable objective.

A stronger paper claim would be something like:

We introduce an adversarial regularization scheme that improves local off-trajectory robustness of Gaussian splatting under sparse trajectory capture.

Not:

We learn correct geometry for unseen viewpoints.

That distinction matters.

If you want, I can help rewrite the method description around this more precise framing so it sounds technically solid and defensible.

What about scored distillation instead of the critic for guiding off-path reconstructions?

Yes — **score distillation is a much more principled alternative than a critic** for this particular role, but it changes what prior you are injecting.

The short version is:

- **Critic** says: “make off-path renders look like on-path renders”
- **Score distillation** says: “make off-path renders look like valid natural images under a pretrained image prior”

That is a real difference.

Why score distillation is attractive here

Your earlier objection to contrastive and energy was exactly right: they do not create new supervision if all you have is on-path vs off-path.

Score distillation does add a new source of information:
a pretrained diffusion model.

So now there really is an external prior. The system is no longer trying to bootstrap everything from trajectory membership alone.

That means for an off-path render with no ground-truth image, you can still ask:

Does this image look like a plausible image of the scene, according to a strong image prior?

That is a more meaningful signal than “does this look on-path?”

But there is a catch

The diffusion prior is usually a prior over **generic images**, not over **your specific scene geometry**.

So score distillation can help with:

- reducing obvious artifacts
- sharpening broken regions
- discouraging implausible textures or blobs

But it can also hurt by:

- hallucinating scene content
- changing appearance in ways unsupported by the observations
- favoring photorealism over geometric correctness

So the question is not “is SDS better?” in the abstract.

The real question is:

Is the failure mode you want to fix more about image plausibility or scene-specific geometric accuracy?

If your off-path renders are failing because they look obviously broken, smeared, or unnatural, SDS could help.

If they fail because the geometry is wrong in a scene-specific way, SDS may not fix that very well, and may even paper over it.

Intuition in your setting

Suppose you shift two meters to the side.

The render may have:

- stretched facades
- torn foliage
- floaters
- bad disocclusions
- splats that were only ever trained from the original lane

A critic trained only on your own data may just learn path bias.

An SDS prior might instead push those images toward something that looks more like a coherent street scene.

That is useful.

But it still does **not know**:

- what building is actually behind the occluder
- what exact texture belongs there
- whether the geometry is truly correct for your scene

So SDS is a **plausibility prior**, not a correctness oracle.

Compared to your critic idea

Critic

Pros:

- scene-specific
- lightweight
- naturally tied to your capture setup

Cons:

- weak supervision
- likely to learn on-path/off-path shortcuts
- no external knowledge

SDS

Pros:

- introduces real external prior
- can regularize off-path views even without labels
- may suppress ugly artifacts better than a critic

Cons:

- can hallucinate
- may fight scene fidelity
- optimization can be noisy and expensive
- hard to preserve identity of a specific place unless strongly anchored

The biggest practical issue

If you use SDS directly on rendered images, the diffusion model will not know your intended camera perturbation is supposed to depict the **same scene** as the captured one.

It just sees an image and pushes it toward the natural image manifold.

So unless you condition it carefully, it may improve realism while drifting away from the actual scene.

That is the central danger.

When SDS makes the most sense here

I would use it as a **weak regularizer**, not the main objective.

So:

- keep on-path reconstruction as the dominant loss
- apply SDS only to small off-path perturbations

- weight it lightly
- keep perturbations local enough that “same scene, nearby view” is a reasonable assumption

That way SDS acts like:

“don’t become obviously implausible when I move a little off the path”

rather than:

“invent whatever looks good from this new angle”

What I would not do

I would not replace the whole method with pure SDS and expect it to recover correct unseen-view geometry. That is too optimistic.

Best interpretation

Given your setup, SDS is appealing because it is one of the few options that truly provides **additional information** beyond trajectory labels.

So unlike the earlier energy/contrastive discussion, this is not just a relabeling of the same weak signal.

But what it provides is:

- a **generic image prior**
not
- a **scene-grounded 3D supervision signal**

That is the right mental model.

My honest take

If your choice is:

- critic trained only on on-path vs off-path
versus
- lightly weighted SDS on off-path renders

then SDS may actually be the more defensible source of supervision, because it injects a genuine external prior instead of relying on a possibly meaningless binary distinction.

But I would trust it only as a regularizer for **local robustness**, not as a way to learn true off-path correctness.

A clean claim would be:

Use score distillation as an off-trajectory image prior to discourage implausible renderings under small viewpoint perturbations.

That is much stronger than claiming it recovers correct unseen geometry.

If you want, I can next compare **critic vs SDS vs both together** in a very concrete loss-function form.

Let's continue this. I've been trying to avoid exposing my student to too much complexity at once by focusing on a version of the project that I thought would be simple to implement and understand. Hence, the critic approach was the first one that I suggested. Score distillation is also good. My experience with score distillation is it's a little bit questionable whether it actually works very well or not. Some people claim to show good results from it, but, you know, kind of my experience with it has been sketchy. Then, of course, the other approach, which has become an option only very recently, is to use public domain open source video models. I think that there are models where you can take an image and render, like, say, take an image and render which way you want to go and render moving through the image that way, and it will use generative modeling, fake a trajectory through the image. I believe that there are open source models that we could use here and use those to generate fake data that's off the path. It's similar to score distillation in a way, but what it is is it's actually creating extra samples to augment our actual training data. That one could also work, and it's, I mean, it's got its pros and cons. In some ways, if you can get your hands on the video model, it's the absolute simplest because then you just use the regular pipeline. I also feel like it's kind of the least interesting, but it does take advantage of the recent advances in video models.

This is a really good place to pause and make a **research decision**, not just a technical one. You've basically laid out three viable directions, and they differ less in implementation difficulty than in **what signal they inject** and **what story they tell**.

Let me frame them cleanly in *your exact context*, and then I'll give you a practical recommendation given your "don't overwhelm the student" constraint.

The Three Options (Reframed)

1. Critic (your original idea)

What it uses:

- Only your data
- Only your perturbations

What signal it injects:

“Off-path renderings should look like on-path renderings”

Strength:

- Conceptually clean
- Fully self-contained
- No external dependencies
- Good pedagogically

Weakness:

- Weak supervision signal
- Easy for critic to learn shortcuts
- May not actually enforce geometry

Research value:

 **High** — it's novel, principled, and directly tied to the problem

2. Score Distillation (SDS)

What it uses:

- External pretrained diffusion prior

What signal it injects:

“Images should lie on the natural image manifold”

Strength:

- Real external signal
- Can suppress obvious artifacts

Weakness:

- Can hallucinate
- Scene identity drift
- No guarantee of geometric correctness
- Optimization instability (you already observed this)

Research value:

⚠ Medium — useful, but feels like “apply a known trick”

3. Video Model Augmentation (your new idea)

What it uses:

- External video model
- Generates synthetic off-path views

What signal it injects:

“Here is what the scene *might* look like from nearby viewpoints”

Strength:

- Turns problem back into **fully supervised learning**
- Dead simple pipeline after data generation
- No adversarial training
- No tricky losses

Weakness:

- Hallucinated ground truth
- Model bias baked into training data
- May produce physically inconsistent views
- Hard to know if improvement is real vs inherited

Research value:

⚠ Medium–low (but very practical)

The Real Tradeoff (This is the key insight)

All three methods differ in **where the “off-path signal” comes from**:

Method	Source of supervision
Critic	Your own renders (self-consistency)
SDS	Image prior (generic realism)
Video	Generated pseudo-ground-truth

So the core question becomes:

Do you want to **regularize**, or do you want to **hallucinate supervision**?

My Honest Take (given your goals)

You said two very important things:

- You want something **simple for a student**
- You also care about it being **interesting (research-wise)**

These are in tension.

If the goal is: “Get something working reliably”

 **Use the video model augmentation**

Because:

- It reduces everything to:

“train Gaussian splats on more views”

- No new optimization tricks
- No adversarial instability
- Easy to debug
- Fast iteration

This is the **engineering-optimal choice**

If the goal is: “Do something intellectually interesting”

👉 **Stick with the critic**

Because:

- It directly addresses the *core problem*
- It's not outsourcing the solution
- It explores:

“Can we enforce 3D consistency without supervision?”

That's a real research question

If the goal is: “Inject stronger signal without too much complexity”

👉 **Hybrid: video model as teacher, but weakly**

This is the interesting middle ground:

- Use video model to generate off-path views
- But **don't fully trust them**
- Use them as:
 - soft targets
 - or consistency constraints

That avoids:

- fully committing to hallucinated ground truth



Important Insight About Video Models

You said:

“It’s the simplest but least interesting”

I’d sharpen that:

It’s simple because it **outsources the hard part of your problem**

Your actual research question is:

“How do we generalize 3D representations beyond observed viewpoints?”

The video model answer is:

“We let a giant pretrained model guess the missing views”

That’s why it feels less interesting — because it kind of is 😊



One Subtle Risk with Video Models

This is important:

If you train splats on generated views:

- The splats may learn:
 - the **biases of the video model**
 - not the true geometry

Then at test time:

- You might look good...
- but only because you're mimicking the generator

Your validation setup (other side of the street) is **really good** though — it will expose this.



Recommendation (Concrete)

If I were guiding a student:

Step 1 (baseline, simple, publishable-ish):

👉 Implement **critic version**

- Keep it minimal
 - Small perturbations
 - Show improvement over vanilla splatting
-

Step 2 (optional extension):

👉 Add **video model augmentation**

- Compare:
 - baseline
 - +critic
 - +video augmentation

Now you have a **clean experimental story**

Step 3 (if needed):

👉 Try SDS lightly

- but don't rely on it
-



Framing for a Paper

You now have a very nice structure:

We study three ways to improve off-trajectory robustness in Gaussian splatting:

- (1) adversarial self-consistency
- (2) generative priors (SDS)
- (3) synthetic view augmentation via video models

That's actually a strong and modern story.



Final Opinion (blunt)

- Your instinct that SDS is “sketchy” → correct
 - Your instinct that video models are “simple but less interesting” → also correct
 - Your original critic idea → still the most *research-aligned* with the problem
-

If you want, next I can help you design the **simplest possible critic setup that avoids the biggest failure modes**, so your student doesn't get stuck in adversarial debugging hell.

what is recommendation critic version, and then these are optional extensions. I agree. Okay, good. So we're on the same page. I think I'm going to want you to recommend the critic setup for my student, but for now, I also want to discuss something else to address in part the instability that you talked about. And the idea here is to have a curriculum. So when we're jittering for off path in the early stages, we might want to jitter by zero meters, so literally off and on path at the, well, okay, not by zero meters. When we're training the critic, we might want to jitter by arbitrary amounts, but when we're fitting the splats, we might want to start with very small jitters and over time gradually increase the amount of the jitter so that the critic can't latch on to the obvious cues because already splats are going to be slightly robust to very small perturbations, I believe. I'm not sure about that.

Yes — that is a **good idea**, and it is one of the most plausible ways to make the critic version behave sensibly.

The short version is:

Curriculum on the splat-side perturbation magnitude makes sense.

It turns the off-path objective from “solve the whole problem immediately” into “be locally robust first, then gradually broaden that robustness region.”

I think that is much more defensible than throwing large perturbations at the splats from the beginning.

Why the curriculum makes sense

Early in training, the splats are bad even **on** the path, or at least only loosely organized. If you ask them to also look good from viewpoints that are 1–2 meters off the path, the adversarial signal may mostly reflect junk:

- missing geometry
- unstable density
- bad occlusion handling
- poorly localized splats

So the off-path loss can become noisy or actively misleading.

A curriculum helps because it first asks for:

- **nearby-view consistency**
- then **slightly broader nearby-view consistency**
- then more substantial off-path robustness

That is a very natural progression.

The important distinction: critic training vs splat training

I think your intuition here is also good:

- the **critic** can probably see a broader range of perturbations
- the **splats** should probably be trained adversarially on a narrower range at first

Those do not have to be the same distribution.

That is a useful degree of freedom.

Why that helps

If the critic only ever sees tiny perturbations, it may learn only a very local notion of “badness.”

If it also sees moderate or large perturbations, it can learn a broader representation of artifact structure.

But for splat updates, you do not necessarily want to optimize against those large perturbations early, because the splats are not ready for them yet.

So conceptually:

- **critic distribution** = broad enough to learn what off-path failure looks like
- **generator/splat distribution** = curriculum, starting very local

That sounds reasonable.

Your core hypothesis

You said:

maybe the critic can't latch onto obvious cues because the splats will already be somewhat robust to small perturbations

That is plausible, but I would phrase it more carefully.

I would not say the curriculum prevents the critic from latching onto shortcuts. The critic may still latch onto shortcuts.

What the curriculum more reliably does is:

it makes the adversarial task for the splats locally achievable.

That matters a lot.

If early perturbations are tiny, then the desired change in the rendered image is also small. So the splats have a chance to improve in a smooth way, instead of getting slammed by a critic on wildly bad off-path renders.

So the main benefit is less "critic becomes pure" and more "optimization becomes sane."

What I think is true physically

For a real static scene, small pose perturbations should usually produce:

- small parallax shifts
- small exposure to disocclusions
- mostly stable appearance

So asking the model to be robust to tiny perturbations first is aligned with the geometry of the problem.

That makes it a good curriculum variable.

A concrete way to think about it

Let the perturbation radius used for adversarial splat updates be $r(t)r(t)r(t)$.

Early:

- $r(t)r(t)r(t)$ very small

Later:

- gradually increase

So the model is learning something like:

first be stable within a tiny tube around the path,
then a wider tube,
then a wider one still.

That is a very intuitive training story.

One caution

I would avoid starting at literally zero for the splat-side adversarial samples.

If the perturbation is exactly zero, then “off-path” equals “on-path,” and the critic objective becomes degenerate for that branch.

Better is:

- start with a **very small but nonzero** perturbation
- maybe a few centimeters or something small relative to scene scale and pose noise

That way the task is distinct, but still gentle.

Another caution: critic distribution mismatch

There is one possible issue in the setup you proposed.

If:

- critic trains on very large jitters
- splats train against only tiny jitters

then the critic may devote most of its capacity to cues relevant for large perturbations, which may not be the most useful signal for tiny ones.

So I would not make the critic distribution arbitrarily broad from the beginning.

A better version is probably:

- critic sees a **mixture**
 - lots of small perturbations
 - some medium perturbations
 - maybe a few large ones
- splats are optimized against
 - mostly small perturbations at first
 - then progressively larger ones

That keeps the critic somewhat aligned with the splat curriculum.

The biggest benefit of curriculum here

I think the biggest benefit is that it changes the claim from:

“make unseen views good”

to:

“make the representation locally stable under small viewpoint deviations, and expand that stability radius over training”

That is a much stronger and more defensible idea.

It also matches your validation story:

- you ultimately care about larger deviations,
- but learning local robustness first is a reasonable route there.

Where it might fail

The curriculum will not solve everything.

Possible remaining issues:

- critic still learns low-level artifacts
- larger jitters later may still expose geometry the model cannot infer
- curriculum may encourage smoothness more than true correctness

But even if that happens, curriculum is still a sensible stabilization tool.

My take

I think this is a **good design choice** and probably one of the best knobs to give your student:

- easy to explain
- easy to implement
- likely to help
- conceptually aligned with the task

So yes, I would absolutely keep this idea in the recommended critic setup.

The clean version would be:

Train the critic on a distribution of off-path perturbations, while training the splats adversarially using a curriculum that starts with very small perturbations and gradually expands the perturbation radius.

That sounds technically coherent.

The one tweak I'd make is:

keep the critic's perturbation distribution partially curriculum-aligned too, rather than making it fully arbitrary from the start.

That should give a cleaner signal.

When you want, the next step is to turn this into a **very concrete student-friendly training recipe**: what gets sampled, what losses are used, and what schedule to use.

There's one other bit of signal that we have, which is to distinguish an actual frame of the video from a rendered splat off-path or on-path. How could that be used to help make the critic better?

Yes — that is **very useful**, and in my opinion it is probably the **most important extra signal** you have beyond pure on-path/off-path labeling.

Because now the critic does not have to learn only:

- on-path render vs off-path render

It can also learn:

- **real image vs rendered image**

That gives it access to an anchor that is tied to actual data, not just trajectory membership.

Why this helps

Your earlier concern was exactly right: if the critic is trained only on

- on-path render
- off-path render

then it may just learn:

“which render came from the camera path distribution?”

That is weak.

But if you also show it **real frames from the video**, then the critic can learn a richer notion of:

“what real, observed imagery looks like”

That matters because it now has a grounded target.

The simplest way to think about it

You now have **three domains**:

1. **Real video frame** at an observed pose
2. **Rendered on-path image** at that same pose
3. **Rendered off-path image** at a perturbed pose

That opens up a few options.

Option 1: Two-head critic

This is probably the cleanest.

Train one shared backbone, but with two heads:

Head A: realism head

Predict:

- real frame
- rendered image

This uses:

- real video frames as positive
- both on-path and off-path renders as negative, or possibly only on-path as the main negative at first

So this head learns:

“is this a real captured image or a splat rendering?”

Head B: robustness head

Predict:

- on-path render
- off-path render

This head learns:

“does this render look like the kind of render we get from stable, observed viewpoints?”

Now the backbone is encouraged to represent both:

- realism relative to the real data
- degradation associated with leaving the path

That is much better than just the second task alone.

Why this is nice

Because it separates two questions:

- **Does it look like a real image at all?**
- **Does it look like a render that is stable near the training trajectory?**

Those are not the same thing.

Option 2: Three-way classifier

Instead of two heads, just classify into:

- real frame
- on-path render
- off-path render

This is simpler to implement.

The critic now has to learn a representation where:

- real frames are the gold standard
- on-path renders should move toward real frames
- off-path renders should ideally move toward on-path and real

This can work, but I think it is a little less interpretable than the two-head version.

Because with a three-way classifier, the geometry of the objective is less controlled. It may spend too much effort separating real from all renders, which is easy and not always useful.

So I slightly prefer the two-head setup.

Option 3: Ranking / relative losses

You can also define the critic so that for a given observed pose:

- real frame should score best
- on-path render should be close
- off-path render should score worse

In other words, enforce something like:

real > on-path > off-path

This is conceptually elegant, but probably a little more complexity than you want right now.

How this helps the splats

This is the important part.

The critic gives two kinds of pressure:

1. On-path realism pressure

Since on-path renders are compared against real frames, the splats get pushed to produce renders that are more photo-consistent.

That complements reconstruction loss.

2. Off-path robustness pressure

Since off-path renders are judged relative to on-path renders and/or real frames, the splats get pushed not just to “look on-path,” but to move toward the actual appearance distribution of captured data.

So the critic is less likely to become a pure path-membership detector.

Very important caveat

You do **not** want the splats to optimize too hard against the “real vs render” signal globally.

Why?

Because rendered images are never going to be literally indistinguishable from real video frames in every low-level sense:

- sensor noise
- exposure
- motion blur
- rolling shutter
- dynamic objects
- compression artifacts

If you optimize too aggressively against that, the critic may focus on nuisance cues, and the splats may chase impossible targets.

So this signal should be used as a **regularizer**, not the dominant objective.

Your main anchor should still be:

- reconstruction on-path

The realism critic should be secondary.

Best use, in my opinion

The best role for real frames is:

calibrate the critic, not fully supervise the off-path views.

That is, real frames help the critic learn what “good imagery” looks like so that its judgment about renders is less arbitrary.

That is stronger than using them as a direct target for off-path training.

Student-friendly recommendation

If I were simplifying this for a student, I would recommend:

Critic input classes

- real frame at pose p
- render at pose p
- render at perturbed pose $p + \Delta$

Critic outputs

Two heads:

Head 1: real vs rendered

- real frame = positive
- on-path render = negative
- off-path render = negative

Head 2: on-path vs off-path

- on-path render = positive
- off-path render = negative

Splat losses

- standard reconstruction loss on-path
- small weighted adversarial loss from head 2 on off-path renders
- optional small weighted realism loss from head 1, mostly on on-path renders

That way:

- the realism head teaches the critic what “actual imagery” looks like
 - the robustness head teaches it what “trajectory-consistent rendering” looks like
 - the splats don’t get dominated by impossible photorealism objectives
-

One more subtle benefit

Using real frames may also help prevent a degenerate critic that just separates two kinds of synthetic images.

Without real frames, the critic lives entirely in render-space.

With real frames, it gets anchored to the actual data manifold, which should make its features more meaningful.

That is a real advantage.

My bottom line

Yes — **real frames should probably be part of the critic training.**

Not because they magically tell you what the off-path view should be, but because they help the critic learn a more grounded notion of image quality and realism.

The clean mental model is:

- **real frames** anchor the critic
- **on-path renders** define what the current model can already explain
- **off-path renders** are the robustness challenge

That is a much better setup than on-path vs off-path alone.

The next design question is whether you want the critic to be:

- a **two-head binary classifier**, or
- a **3-way classifier**

My instinct is the two-head version is better for a student and easier to reason about.

You mention a two-head version, but isn't a head only in the output? Wouldn't the inputs be different if you're feeding it the real image versus the on-path splat rendered image versus the off-path splat rendered image? Wouldn't this have to be two distinct classifiers? I mean, two distinct passes at least through the classifier?

Yes — exactly. You're right, and the confusion is just terminology.

A **two-head critic** does **not** mean the inputs are somehow combined into one tensor or that the network sees all three images at once in a single forward pass.

It means:

- you have **one shared backbone**
- and **two output heads**
- and you run **separate forward passes** for each image you want to score

So in practice you would absolutely do distinct evaluations of:

- real image
- on-path render
- off-path render

through the same network.

What “two-head” actually means

Suppose the network is:

- backbone $f(x) \rightarrow$ feature vector
- head A: realism
- head B: path-consistency

Then for each image x , you compute:

$$h = f(x)$$

and then:

- $A(h)$: real vs rendered
- $B(h)$: on-path vs off-path

So if you have three images:

- x_{real}
- x_{on}
- x_{off}

you do three forward passes:

- $f(x_{\text{real}})$
- $f(x_{\text{on}})$
- $f(x_{\text{off}})$

and each one can produce both outputs.

So your statement is correct

“Wouldn’t this have to be two distinct passes at least through the classifier?”

Yes.

Usually **three passes**, one per image.

The “two-head” part refers only to:

- the network has two prediction branches at the output

not to the number of inputs per pass.

Why use one shared network at all?

The reason to share the backbone is that both tasks may benefit from similar visual features:

- texture quality
- ghosting
- blur
- edge artifacts
- shape distortions
- disocclusion artifacts

So instead of training:

- one classifier for real vs rendered
- another completely separate classifier for on-path vs off-path

you can train:

- one feature extractor
- two lightweight output heads

That is the standard meaning.

Would two separate classifiers also work?

Yes, absolutely.

And honestly, for a student, **two separate classifiers might even be conceptually cleaner.**

Because then you can say:

Classifier 1

Input: one image

Output: real or rendered

Classifier 2

Input: one image

Output: on-path or off-path

That is extremely easy to understand.

The downside is:

- more parameters
- less shared learning
- slightly more engineering

But conceptually it is perfectly valid.

The real design choice

So the question is not:

“Can two-head work if the inputs are different?”

Yes, because the inputs are still just individual images processed one at a time.

The real choice is:

Shared-backbone, two-head model

Pros:

- parameter sharing
- cleaner joint learning
- more efficient

Cons:

- slightly more abstract to explain

Two separate classifiers

Pros:

- simpler mental model
- easier to debug conceptually

Cons:

- redundant computation
 - less shared representation
-

What I would recommend for your student

Given everything you said about keeping complexity down, I would actually lean toward:

start by thinking of them as two separate classifiers, even if implementation later shares a backbone.

Because pedagogically that is much cleaner.

You can explain it as:

- one classifier teaches “does this look real?”
- one classifier teaches “does this look like a stable in-distribution render?”

Then later, if desired, you can merge them into one shared backbone with two heads.

One more subtle point

If you use the “real vs rendered” classifier, be careful:

- real images contain sensor noise, motion blur, exposure variation, dynamic objects
- renders do not

So that classifier may learn easy nuisance cues.

That does not make it useless, but it means I would treat it as a **supporting signal**, not the main one.

The main classifier for your method is still probably:

- **on-path render vs off-path render**

with the real-vs-rendered classifier acting as an auxiliary stabilizer.

Bottom line

You are correct:

- the images are still fed separately
- there would be separate forward passes for real, on-path, and off-path images
- “two-head” only means the network has two output branches, not that all images are processed together

So yes, if the terminology is getting in the way, it is totally fine to just think of this as:

two classifiers, each run independently on each image

That is a perfectly accurate mental model.

Maybe it's simpler to let the on-path MSE be our only anchor for realism and focus on on-path versus off-path for now with a single-headed classifier, with the realism being a possible optional extension later.

Yes — I think that is the right call.

That is the cleanest version of the project, and it matches your goal of not overloading the student.

Why this is the simpler and better starting point

You already have a strong realism anchor:

- **on-path reconstruction loss**

That is grounded in actual observations and easy to explain.

So if you also add a real-vs-rendered classifier, you are introducing another notion of realism that is:

- noisier
- easier to game with nuisance cues
- harder to explain cleanly

By contrast, the single-headed critic gives a very simple story:

- on-path MSE says: **match what we observed**
- critic says: **do not fall apart when we move slightly off the path**

That is a coherent division of labor.

Why this is good for the student

It keeps the method understandable:

- one reconstruction loss
- one critic loss
- one perturbation curriculum

That is enough novelty already.

It also makes debugging easier. If something goes wrong, there are fewer moving parts:

- perturbation magnitude
- critic strength
- critic architecture
- mixing weight with MSE
- curriculum schedule

That is manageable.

Conceptual framing

I would frame it like this:

The reconstruction loss enforces fidelity on the observed camera trajectory.

The critic regularizes the splats so that renderings remain plausible under nearby viewpoint perturbations.

That is clean and defensible.

What you are giving up

You lose the extra signal from real-vs-rendered discrimination.

But I think that is fine for version 1, because that signal is optional and somewhat ambiguous anyway. It may help later, but it is not necessary to test the core idea.

What I would recommend as the base project

For the first implementation:

- **On-path loss:** MSE or L1, whatever your baseline splatting pipeline already uses
- **Off-path loss:** single-head classifier distinguishing on-path render vs off-path render
- **Splat objective:** reconstruction on-path plus adversarial loss on off-path
- **Curriculum:** start with very small off-path jitters and grow over time
- **Validation:** held-out return path on the other side of the street

That is already a complete and interesting project.

Why this is probably the best first paper story too

Because it isolates the main question:

Can an adversarial off-path regularizer improve local view robustness in Gaussian splatting?

If you add too many extras at the beginning, it becomes harder to know what actually mattered.

So yes — I would absolutely recommend:

Use on-path MSE as the only realism anchor, and use a single-headed on-path vs off-path critic for the first version.

That sounds like the right baseline method.

When you're ready, the next step is to turn this into the concrete recommended setup for your student: exact sampling, exact losses, and the curriculum schedule.

Okay, so you don't have the context, but we're doing Gaussian splatting, which Gaussian splatting usually tries to predict where the Gaussian splats are to minimize the mean squared

error between the reconstructed and then the viewpoints that you've got actual imagery for. But we have sparse imagery. And so what we want to do is we want to use a critic that we train to predict whether we're at a perturbed new location or one of the original locations. And so it can evaluate on-path, because the original frames are taken on a path, so it can evaluate on-path imagery versus off-path perturbed imagery rendered from the splat. And the idea is there will be that critic, which is an adversarial score, and we'll use that as a part of the loss when we fit the splats so that the idea is the Gaussian splat should look realistic even off the splats. So that's the idea. But then the question is, what should the structure of the critic be? And so we've got a rendered image that was rendered slightly off the path, and we can, as training goes, go further and further off the path as a part of a learning curriculum. But for that image that was captured slightly off the path, but we also have images that are on the path, but they had dynamic objects like moving cars, pedestrians, bicycles, or the actual structure connected to the GoPro that took the pictures. Those things we can mask out even when we're on the path. So what should the structure of a critic be? The critic shouldn't be, for regularization purposes, the critic should be pretty simple. We don't want it to learn faster than we fit the Gaussian splats. So I'm interested in sort of this patch-based critic idea, which is actually, we take a simple CNN, I don't know whether VGG-style, but with fewer classification heads, or a ResNet, or what, but just some sort of a very or custom simple CNN, but the idea is it probably down-samples the image a few times, and so then basically we have one-thirty-second the number of pixels, like a small number of pixels, and every pixel should be separately predicting, is this pixel from an on-path or an off-path? That way, if we mask out objects, the supervision for the critic cannot just be, was this image from on-path or off-path, but also, if you're on-path, was this pixel masked out because something dynamic was there or not? So that way, the dynamic stuff, we're not even doing the MSE or anything when we fit the splats. We just want the splats to interpolate those the same way that you would off-path with realistic stuff. So explain that and give me guidance on whether this structure of a critic for the GAN makes sense.

Yes — the **patch critic / PatchGAN-style discriminator** is a very sensible structure for what you want.

What you are really trying to do is not “is this whole image real or fake?” in the classic GAN sense. You are trying to ask:

1. **Does this rendered appearance look like something that could occur along the true data manifold of the route?**
2. **Does it still look plausible when I move slightly off the training path?**
3. **Can I ignore dynamic-object regions and other nuisance regions so the critic does not punish the splats for things they should not model?**

A small patch-based critic lines up well with all three.

First, restating the setup in learning terms

You have:

- **On-path real images** from the camera trajectory.
- **Rendered images from the Gaussian splat model** at:
 - on-path viewpoints
 - slightly perturbed off-path viewpoints
 - possibly farther off-path viewpoints later in training via curriculum
- **Masks** for dynamic / nuisance regions in the real images:
 - cars
 - pedestrians
 - bicycles
 - camera rig / GoPro mount
 - anything else you do not want the splat field to explain literally

Your ordinary reconstruction term handles the places where you truly have supervision. But because views are sparse, MSE alone encourages the splats to do whatever minimizes training-view error, even if the geometry or appearance becomes brittle off-path. So the critic is acting as a **manifold prior**: “off-path renderings should still look like route-consistent imagery.”

That is a good use of an adversarial term.

Why a patch critic makes more sense than a global image critic

A global image-level critic would be weaker for your case.

Why:

- The difference between on-path and slightly off-path is often **local**, not global.
- Realism failures in Gaussian splatting usually show up as:
 - stretched texture
 - floaters
 - ghosting
 - double edges
 - inconsistent parallax
 - local blur or local hallucinations
- Dynamic masks are naturally **spatial**, not image-level.
- A global critic can cheat by using crude signals:
 - horizon shift
 - camera pose statistics
 - overall framing bias
 - route-specific layout cues

A patch critic forces the model to care about **local plausibility**. That is usually exactly what you want when supervising rendered images.

So your idea that the critic outputs a low-resolution spatial grid, where each output cell judges a receptive field, is the right starting point.

The most important adjustment: do not frame it as “on-path vs off-path” only

This is the place where I would be most careful.

If the critic is trained literally to answer:

- 1 = on-path
- 0 = off-path

then it may learn the wrong thing.

It may learn viewpoint statistics rather than realism. For example, it might detect that off-path views systematically have different perspective cues, crop statistics, or parallax patterns, even when they are perfectly plausible. Then the generator is pressured not to make images realistic, but to make them look more like the training path distribution.

That is too restrictive.

What you really want is closer to:

- **real on-path valid content**
- vs
- **rendered content that departs from the real route-image manifold**

So I would recommend one of these formulations.

Best formulation: realism critic, not route-membership critic

Train the critic to distinguish:

- **real image patches from unmasked on-path data** as positive
- **rendered patches from the splat model** as negative

Then sample rendered patches from:

- on-path renders

- near off-path renders
- farther off-path renders later in curriculum

This way the critic is learning **real vs rendered**, not **on-path vs off-path**.

That is much safer.

If you want explicit off-path awareness

Condition the critic on pose perturbation magnitude, or split its outputs into:

- realism score
- perturbation-awareness score

But I would still keep the main adversarial pressure as **realistic vs unrealistic**, not **on-path vs off-path**.

Because off-path should not automatically mean “bad.” It should mean “less directly supervised, so test realism more carefully.”

Recommended critic structure

A very small **PatchGAN-style CNN** is probably the right bias.

Good default architecture

Use a simple fully convolutional network:

- input: RGB image, optionally plus mask channels and maybe pose-conditioning channels
- 4 to 5 convolutional stages
- stride-2 downsampling a few times
- small channel counts
- LeakyReLU
- no huge residual trunk
- final 1-channel spatial output map

For example:

- Conv 4x4, stride 2, 32 channels
- Conv 4x4, stride 2, 64 channels
- Conv 4x4, stride 2, 128 channels
- Conv 4x4, stride 2, 128 channels

- Conv 3x3, stride 1, 1 channel output map

That gives a patch score map rather than a single scalar.

This is more appropriate than a ResNet classifier if your goal is “regularizer that does not outrun the splat optimizer.”

Why not ResNet or VGG?

You *can* use them, but I would not start there.

They are often too strong for this role, especially when:

- the real dataset is narrow and route-specific
- the synthetic renders initially have obvious artifacts
- you want the critic to regularize, not dominate

A lightweight CNN is more stable and easier to tune.

What each spatial output should predict

Your instinct is good: each output location should judge the local patch covered by its receptive field.

But I would not make it predict just:

- on-path vs off-path

Instead, I would make it predict something like:

- **valid real patch vs rendered patch**

with masking so only meaningful spatial locations contribute.

If you want richer supervision, a clean way is a **3-way or masked binary setup**:

Option A: masked binary critic

For each output cell:

- positive target: real, unmasked patch from on-path image
- negative target: rendered patch
- ignored: any cell overlapping masked dynamic content in the real image

This is simplest and strong.

Option B: 3-way classification

Per patch:

- class 1: real static on-path
- class 2: masked / dynamic / invalid
- class 3: rendered

This can work, but I think it is more complication than you need unless masks are noisy or large.

Option C: dual-head critic

Shared CNN trunk, then:

- head 1: realism head (real static vs rendered)
- head 2: validity head (static-valid vs masked-dynamic)

This is a nice compromise if you want the critic itself to learn where supervision is trustworthy.

Still, the simplest thing that fits your description is:

Patch critic + explicit loss mask on critic outputs

That is probably the best first version.

How masking should work

This is important.

You said that when an on-path image has a car or pedestrian or camera rig, you want the splat not to be forced to reconstruct that exactly, and you want off-path plausibility to behave similarly.

That suggests two principles:

1. Do not use masked regions as positive “real” evidence

If a real patch contains masked content, it should not teach the critic what “good static structure” looks like.

So in the critic loss, for real images:

- compute the patch-level validity mask

- drop any output cells whose receptive field overlaps masked pixels too much

2. Do not let the critic over-penalize generator outputs in those uncertain regions

If a rendered patch corresponds to a region that is unsupervised or semantically ambiguous because source data were masked, you may also want to reduce adversarial weight there.

This depends on whether you can project mask uncertainty into the rendered view. If you can, great. If not, at least mask the real-side supervision. That already helps a lot.

A practical method:

- downsample the pixel mask to the critic output resolution
 - use average pooling or max pooling depending on how conservative you want to be
 - if masked fraction in a patch > threshold, ignore that patch in the critic loss
-

Curriculum over perturbation distance makes sense

Yes, this part also makes sense.

If you immediately render far off-path early in training, the critic may mostly learn:

- “all off-path renders are fake”
and the generator may receive noisy gradients.

A better schedule is:

- early: on-path + tiny pose perturbations
- middle: moderate perturbations
- later: larger perturbations, but still within a realistic neighborhood of the route

This is essentially a **viewpoint extrapolation curriculum**.

I would keep the perturbations physically meaningful:

- small lateral offsets
- mild yaw changes
- maybe small height changes if plausible
- avoid huge perturbations that produce views no real nearby camera could have seen

Otherwise the critic starts modeling “distribution shift outside the intended deployment regime.”

One subtle issue: the critic can still learn pose bias

Even with a patch critic, it may still exploit pose-correlated cues.

For example:

- off-path perturbations may create characteristic occlusion boundaries
- image borders may shift systematically
- perturbations may expose geometry not seen in training

Some of that is legitimate. Some is shortcut learning.

A few ways to reduce this:

Use both on-path and off-path renders as negatives

Do not let “off-path” be the only negative category.

Include:

- on-path renders from the current splat model
- off-path renders

Then the critic cannot simply learn “off-path = fake.” It must learn “rendered artifacts = fake.”

Randomize crops

Feed crops rather than full images, or at least mix crop positions aggressively, so the critic cannot overuse global frame layout.

Keep receptive field moderate

Patch size should be big enough to see local structure, but not so big that it becomes a whole-image route classifier.

A receptive field corresponding to something like 32–96 pixels is often a reasonable zone to start from, depending on image resolution.

A concrete loss structure that fits your goal

A good baseline would be:

Generator / splat optimization

$$L_G = L_{recon} + \lambda_{adv} L_{adv} + \lambda_{reg} L_{other}$$

Where:

- L_{recon} : photometric loss on valid on-path pixels only
- L_{adv} : fool the patch critic on rendered images
- L_{other} : your normal splat regularizers, depth/opacity/scale/etc.

Critic optimization

Train critic on:

- positive: real on-path unmasked patches
- negative: rendered patches from on-path and perturbed views

Use a stable adversarial loss:

- hinge loss is a strong default
- non-saturating logistic is also fine

Hinge is simple:

$$L_D = \mathbb{E}[\max(0, 1 - D(x_{real}))] + \mathbb{E}[\max(0, 1 + D(x_{render}))].$$

Apply this **per patch**, then average only over valid patch locations.

Generator adversarial term:

$$L_{adv} = -\mathbb{E}[D(x_{render})]$$

again patchwise.

Should the critic see pose information?

Maybe, but I would start **without** it.

If you feed the critic the camera perturbation magnitude or camera pose, two things can happen:

- good: it learns different tolerance at different perturbation distances

- bad: it becomes an explicit pose classifier and shortcuts the task

Given your stated goal of keeping the critic weak and regularizing rather than dominating, I would start with:

- image only
- maybe mask channel if useful
- no explicit pose

Then add conditioning only if you find instability across the curriculum.

Should you compare real on-path and rendered off-path directly?

Not patchwise in a paired sense. There is no true target off-path image.

So the critic should provide an **unpaired realism signal**, not a paired supervision signal.

That is why the PatchGAN framing is appropriate. It says:

- “this patch looks like something from the dataset”
not
- “this patch matches what exactly would have been seen here”

That distinction matters a lot.

Biggest risks with this idea

There are a few.

1. Over-regularizing toward the training-path image distribution

If the critic is too strong, the splat may become overly texture-prior-driven and sacrifice geometry.

Mitigations:

- small critic
- low adversarial weight
- delayed adversarial onset

- slower critic updates or fewer critic steps

2. Critic outruns generator

You already anticipated this correctly.

Mitigations:

- tiny CNN
- spectral normalization or gradient penalty only if needed
- fewer channels
- maybe update critic less often than generator
- keep dataset augmentation modest but present

3. Dynamic masks leaking label information

If masked regions are common only in real images, the critic may exploit mask boundaries or statistics.

Mitigation:

- do not feed raw masks in a way that makes “real image has mask artifacts” a shortcut
- if masks are input, be consistent between real and rendered paths
- better yet, use masks mainly for **loss weighting**, not as a dominant input cue

4. Off-path views becoming semantically unconstrained

If you push too far off-path, “realistic” can become ambiguous. The critic may encourage bland textures rather than correct geometry.

Mitigation:

- restrict perturbation envelope
- keep some geometric prior / consistency term
- curriculum should stop where the task remains meaningful

My recommended first implementation

If I were designing the first version, I would do this:

Critic

- PatchGAN-style fully conv network

- 4 downsampling stages
- modest channels: 32, 64, 128, 128
- LeakyReLU
- final spatial score map
- no big ResNet backbone

Positives

- real on-path images
- only unmasked static regions contribute to the loss

Negatives

- rendered on-path images
- rendered perturbed off-path images

Loss

- hinge GAN loss
- patchwise masking at output resolution

Curriculum

- start adversarial training only after reconstructions are reasonably stable
- start with on-path + tiny perturbations
- grow perturbation radius slowly

Stabilization

- low adversarial weight
- maybe one critic step per generator step, but with much smaller network
- possibly reduce critic learning rate relative to splat optimization

This is simple, aligned with your objective, and unlikely to overpower training.

My main conceptual recommendation

The strongest recommendation I would give is:

Do not make the critic's primary job "classify on-path vs off-path." Make its job "classify real static route imagery vs rendered imagery," while sampling rendered imagery both on and off the path.

That preserves the good part of your intuition while avoiding the main failure mode.

Because the real desired statement is:

- off-path views should still look plausible

not:

- off-path views should look indistinguishable from on-path views

Those are different.

A useful mental model

Think of the critic as enforcing a **local natural-image-and-route prior** over rendered patches.

It should say:

- road edges should look road-like
- facades should stay coherent
- vegetation should not smear
- occlusion boundaries should not ghost
- static content should stay plausible under small pose shifts

It should **not** say:

- only viewpoints from the exact capture manifold are acceptable

That is the line to protect.

Bottom line

Yes, your proposed structure basically makes sense, with one modification:

- **Patch-based CNN critic:** yes
- **Simple, weak architecture:** yes
- **Patchwise outputs at low resolution:** yes
- **Mask-aware supervision:** yes
- **Curriculum over perturbation distance:** yes
- **Train it as on-path vs off-path classifier:** I would change this

- **Train it as real static patches vs rendered patches, with off-path renders included among negatives: this is the better formulation**

If you want, I can turn this into a concrete training recipe with pseudocode and a specific critic architecture.

Okay, the other thing is how we mask the objects out. So right now we're using Mask2Former in order to determine that certain categories of objects should always be removed, like cars and bicycles. The issue is my student is concerned that if we remove a car that's parked, since it's not moving, there'll be no viewpoint where we know what to fill in that hole with. Now, first of all, that's one thing that the critic is supposed to solve eventually, right? So we would know what to fill in that masked hole with according to the critic. But also in the near term, another idea is that we don't remove objects just based off of their category. We also try to detect whether they're moving or not. So in order to detect whether something is moving, I was going to explain that we would use an optical flow algorithm, like the RAFT algorithm. But he doesn't speak English as a first language, and he can't even tell what words I'm trying to say. So can you please explain this notion of using the optical flow algorithms to detect what's moving?

Got it — I'll explain this in a way that's conceptually clear and avoids jargon, so it's easier to communicate.

The core idea (simple intuition)

Instead of removing **all cars**, you only want to remove **cars that are moving**.

To do that, you need a way to answer:

“Did this object change position between two frames?”

That's exactly what **optical flow** tells you.

What optical flow is (very simply)

Imagine you have **two images taken close in time**:

- Frame 1
- Frame 2

Optical flow computes, for **every pixel**, a small arrow:

“This pixel moved from here → to there”

So the output is like a **motion field** over the image.

Intuition example

Let's say you have:

- A parked car
- A moving car
- A building

Between frame 1 and frame 2:

- The **building** doesn't move (only appears to move slightly due to camera motion)
- The **parked car** behaves like the building
- The **moving car** moves differently

So:

- Background + parked car → motion consistent with camera movement
 - Moving car → motion inconsistent (extra movement)
-

The key idea for detecting motion

You already know the camera is moving (because it's on a path).

So:

Step 1: Estimate expected motion from the camera

If the world were completely static, all pixels would move in a **consistent way**.

Step 2: Compare actual motion vs expected motion

- If a pixel follows the expected motion → **static**
 - If it deviates → **moving object**
-

Even simpler explanation (no geometry needed)

You can explain it like this:

“If something stays in the same place in the world, it moves smoothly with everything else when the camera moves.
But if something moves on its own, it will move differently from everything else.”

How RAFT fits in

RAFT is just a **tool that computes optical flow**.

You don’t need to explain the internal details. Just say:

“RAFT takes two images and tells us how every pixel moved.”

That’s enough.

How to turn this into a moving-object mask

Here’s the practical pipeline:

1. Take two nearby frames

- Frame t
- Frame $t+1$

2. Compute optical flow

- For every pixel, get motion vector

3. Find “inconsistent motion”

Now you detect pixels where motion is unusual.

There are two ways:

Option A (simplest): Compare to neighbors

- Most pixels (background) move similarly

- If a pixel moves very differently from nearby pixels → likely moving object

So:

“If this pixel’s motion is very different from its neighbors, mark it as moving.”

Option B (better): Compare to global motion

- Estimate the dominant motion of the scene
 - Pixels that don’t follow it → moving
-

Combining with Mask2Former

Now you combine:

Mask2Former gives:

- “This is a car”

Optical flow gives:

- “This is moving” or “this is static”
-

Final decision:

Object type	Moving?	Action
Car	Moving	Remove
Car	Static	Keep
Pedestrian	Moving	Remove
Bicycle	Moving	Remove

This solves your student’s concern:

A parked car is **not removed**, so you don't create holes you can't fill.

Why this is better than category-only masking

If you remove all cars:

- You create holes where you have **no data**
- The critic must hallucinate everything → unstable

If you remove only moving objects:

- Static objects (parked cars, signs, etc.) stay
 - You preserve real geometry where possible
-

How to explain this to your student (simple script)

You could say something like:

“We have two images. We look at how every pixel moves between them.
If something moves the same way as the background, it is not moving — it is static.
If something moves differently, it is moving — like a car driving.
We only remove objects that are moving.”

One important caveat

Camera motion itself causes everything to move in the image.

So “moving” does NOT mean:

- pixel moved

It means:

- pixel moved **differently than expected**

That's the key concept.

Why this helps your Gaussian splatting setup

- You avoid removing useful static structure
 - You reduce hallucination burden on the critic
 - You improve geometry consistency
 - You still eliminate dynamic artifacts that break reconstruction
-

Final takeaway

- Optical flow = “how pixels move between frames”
 - Static objects follow camera motion
 - Moving objects deviate from it
 - Use that difference to decide what to mask
-

If you want, I can give you a **very concrete implementation recipe** (thresholds, how to compute deviation, how to integrate with masks), or even a diagram you can show your student.